

Introduction to Regression

Part A - kNN



Pavlos Protopapas, Kevin Rader and Chris Gumb

Aseel Rawashdeh
colchuck lake

Lecture Outline

Part A: Statistical Modeling

k-Nearest Neighbors (kNN)

Part B: Model Fitness

How does the model perform predicting?

How do we choose from two different models?

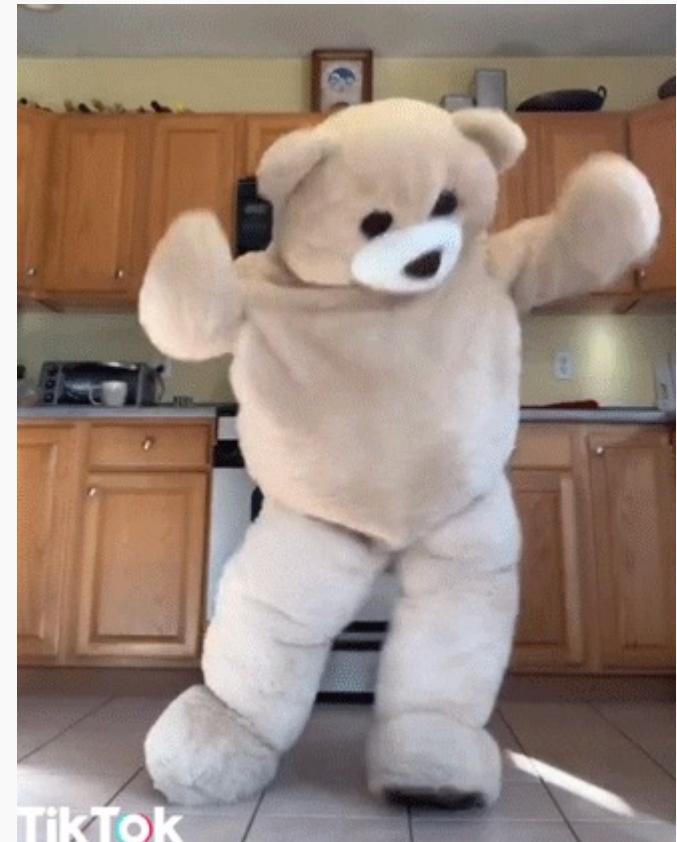
Response and Predictor Variables

Predicting a Variable

Let's consider a scenario in which we aim to **predict** the value of one variable based on another variable or a set of other variables.

Examples:

Predicting the number of views that a **TikTok** video will receive next week, based on factors such as **video length**, **posting date**, and **previous view count**.



Predicting a Variable

Let's consider a scenario in which we aim to **predict** the value of one variable based on another variable or a set of other variables.

Examples:

Forecasting **which movies**, a **Netflix** user is likely to rate highly, considering their **previous movie ratings** and **demographic data**.



Working example

The [Advertising dataset](#) contains sales (in 1000 units) data for a specific product across 200 different markets. It also includes advertising budgets in \$1000 allocated to three different media channels: *TV*, *radio*, and *newspaper*, for each of those markets.

TV	radio	newspaper	sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5
180.8	10.8	58.4	12.9

Some of the figures in this presentation are taken from "An Introduction to Statistical Learning, with applications in R" (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani "

Response vs. Predictor Variables

Many of these problems exhibit an **asymmetry**: the variable we aim to predict may be **harder to measure**, more **significant**, or **directly or indirectly influenced** by other variables.

Therefore, we can classify variables into two categories:

- Variables whose values we aim to **predict**
- Variables **used** as inputs to inform our prediction

Response vs. Predictor Variables

X
predictors
features
covariates
independent variable

y
outcome
response variable
dependent variable

n observations

TV	radio	newspaper	sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5
180.8	10.8	58.4	12.9

p predictors

Response vs. Predictor Variables

$X = X_1, \dots, X_p$
 $X_j = x_{1j}, \dots, x_{ij}, \dots, x_{nj}$
predictors
features
covariates
independent variable

$y = y_1, \dots, y_i, \dots, y_n$
outcome
response variable
dependent variable

n observations

TV	radio	newspaper	sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5
180.8	10.8	58.4	12.9

p predictors

Response vs. Predictor Variables

This is called X : a.k.a.
The Design Matrix

n observations

TV	radio	newspaper
230.1	37.8	69.2
44.5	39.3	45.1
17.2	45.9	69.3
151.5	41.3	58.5
180.8	10.8	58.4

y :
The response variable

n observations	sales
	22.1
	10.4
	9.3
	18.5
	12.9

Response vs. Predictor Variables

This is called X : a.k.a.
The Design Matrix

n observations

TV	radio	newspaper
230.1	37.8	69.2
44.5	39.3	45.1
17.2	45.9	69.3
151.5	41.3	58.5
180.8	10.8	58.4

y :
The response variable

sales
22.1
10.4
9.3
18.5
12.9

Capital letters mean **matrices**,

Response vs. Predictor Variables

This is called X : a.k.a.
The Design Matrix

n observations

TV	radio	newspaper
230.1	37.8	69.2
44.5	39.3	45.1
17.2	45.9	69.3
151.5	41.3	58.5
180.8	10.8	58.4

y
The response variable

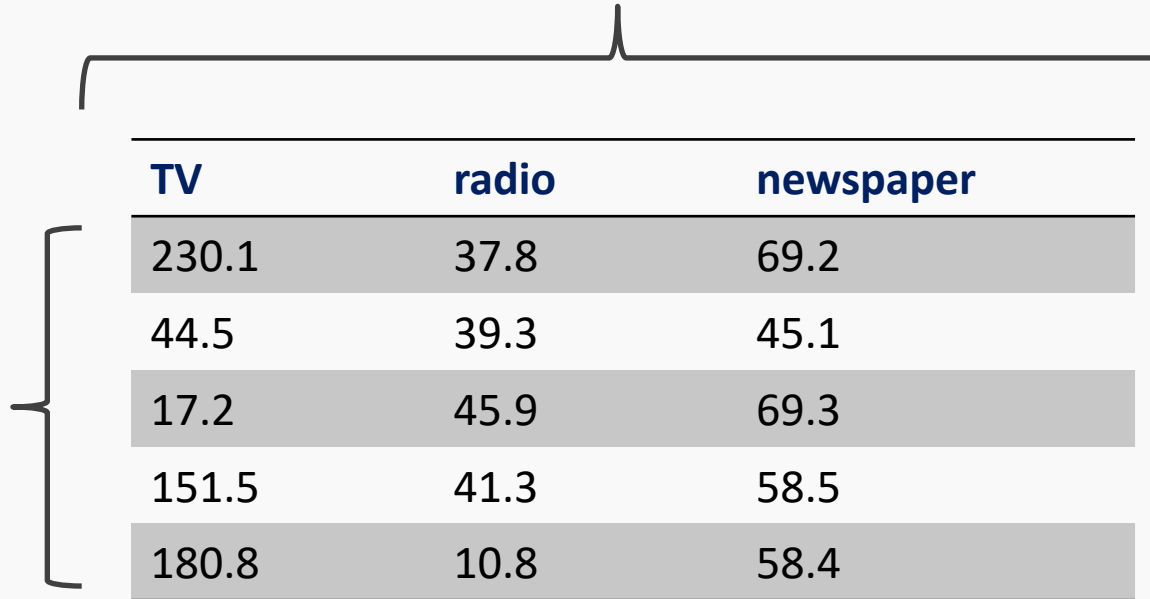
sales
22.1
10.4
9.3
18.5
12.9

Capital letters mean **matrices**, lower case letters mean **vectors**

Sklearn expects certain dimensions

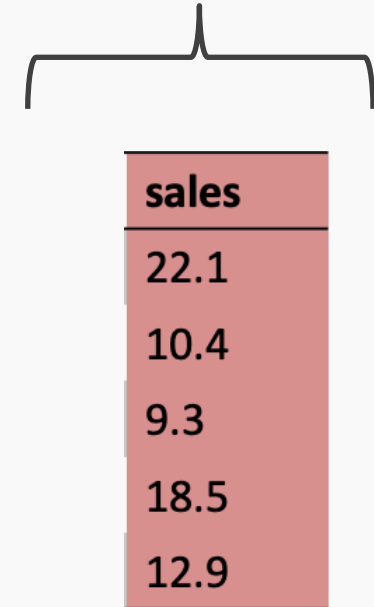
```
>>> X.shape  
(n, p)
```

n observations



TV	radio	newspaper
230.1	37.8	69.2
44.5	39.3	45.1
17.2	45.9	69.3
151.5	41.3	58.5
180.8	10.8	58.4

```
>>> y.shape  
(n,) OR (n, 1)
```



sales
22.1
10.4
9.3
18.5
12.9

Sklearn expects certain dimensions

```
>>> X.shape  
(n, ) OR (n, 1)
```

n observations

TV
230.1
44.5
17.2
151.5
180.8

```
>>> y.shape  
(n, ) OR (n, 1)
```

sales
22.1
10.4
9.3
18.5
12.9

Statistical Model

True vs. Statistical Model

- Imagine an ice cream cone so perfect that it captures every flavor, topping, and swirl of deliciousness. That is what a *true model* is.



True vs. Statistical Model

- Imagine an ice cream cone so perfect that it captures every flavor, topping, and swirl of deliciousness. That is what a *true model* is.
- But reality is like an ice cream shop with infinite flavors and toppings. Trying to fit all of them into one cone is **impossible**.



True vs. Statistical Model

- Imagine an ice cream cone so perfect that it captures every flavor, topping, and swirl of deliciousness. That is what a *true model* is.
- But reality is like an ice cream shop with infinite flavors and toppings. Trying to fit all of them into one cone is **impossible**.
- This is why we use *statistical models*: instead of trying to scoop the impossible sundae, we craft a tasty treat from the flavors we have.



True vs. Statistical Model

We assume that the response variable, Y , is related to the predictor variables, X , through an **unknown function** which can be generally expressed as:

$$Y = f(X) + \varepsilon$$

Here, f represents the unknown function expressing an underlying rule for relating Y to X . ε **represents the random amount** (unrelated to X) that Y differs from the rule $f(X)$.

A ***statistical model*** is any algorithm used to estimate f . We denote the estimated function as $\hat{f}$.

Prediction vs. Estimation

Inference Problems:

- The primary focus is on **obtaining $\hat{f}$** , which is an estimate of the true function f
- Objective: Understand the form and characteristics of $\hat{f}$.



Prediction Problems

- The specific form of $\hat{f}$ is less important than the **accuracy** of the predictions.
- Objective: Minimize the difference between predicted values $\hat{y}$ and observed values y .



Prediction vs. Estimation

Inference Problems

Parametric models (few interpretable parameters)



Examples:

- Linear Regression
- Logistic Regression

Prediction Problems

Non-parametric / flexible models (focus on accuracy)



Examples:

- K-Nearest Neighbors
- Random Forest
- Neural Networks (*parametric but high-dimensional, used mainly for prediction*)

Prediction vs. Estimation

In this lecture, we will be talking about **K-Nearest Neighbors**
(*Prediction, Non-Parametric*)

Why?

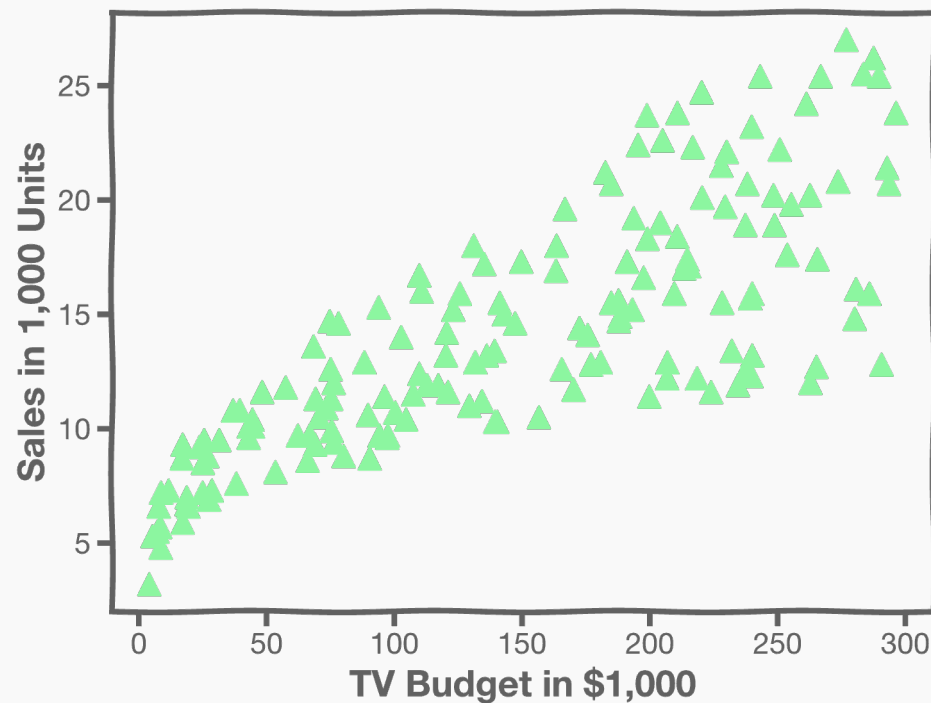


Well... because it's easier to understand.

Example: predicting sales

Motivation: Predict Sales

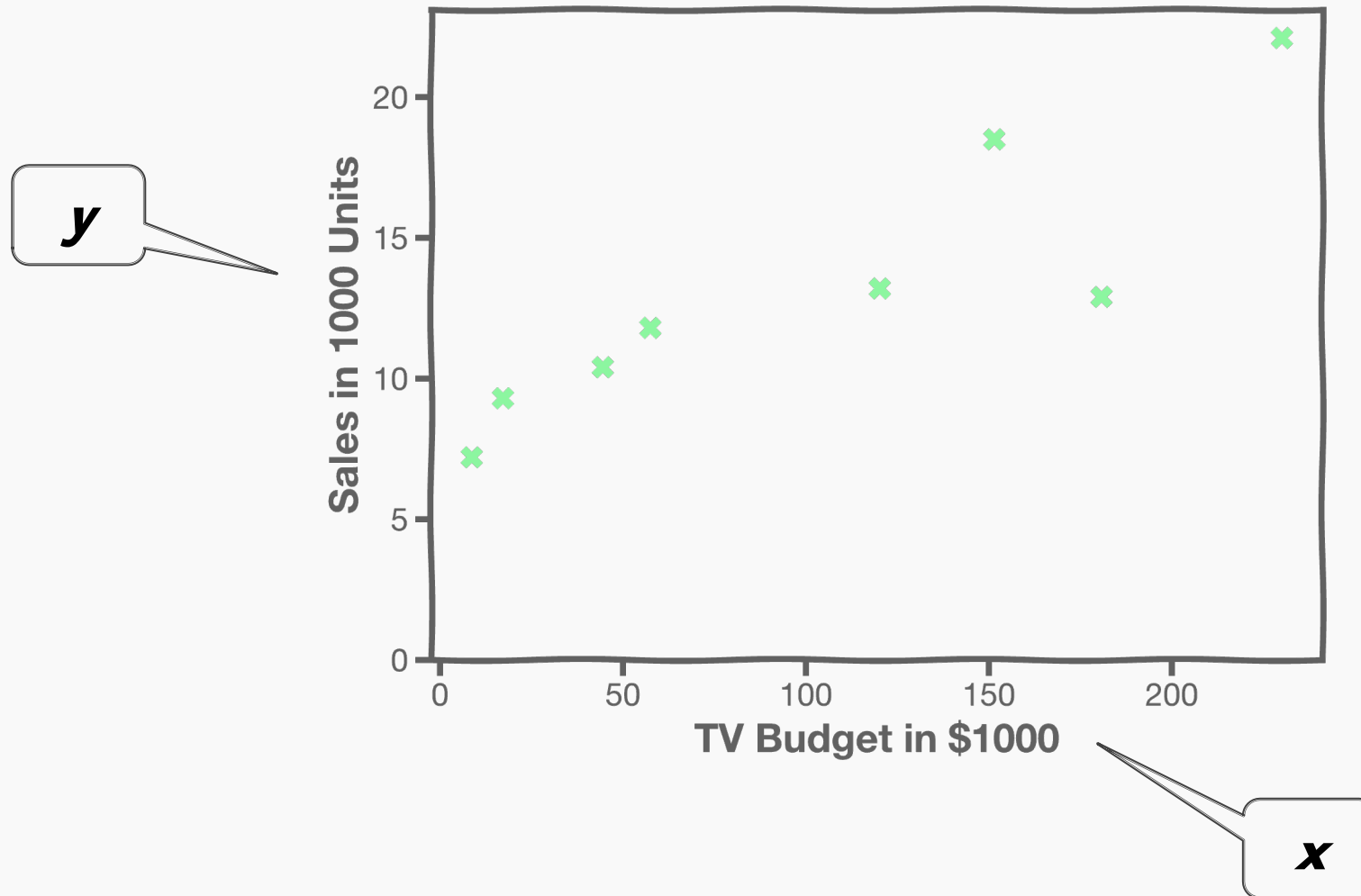
Build a model to **predict** sales based on TV budget



The response, **y**, is the sales.

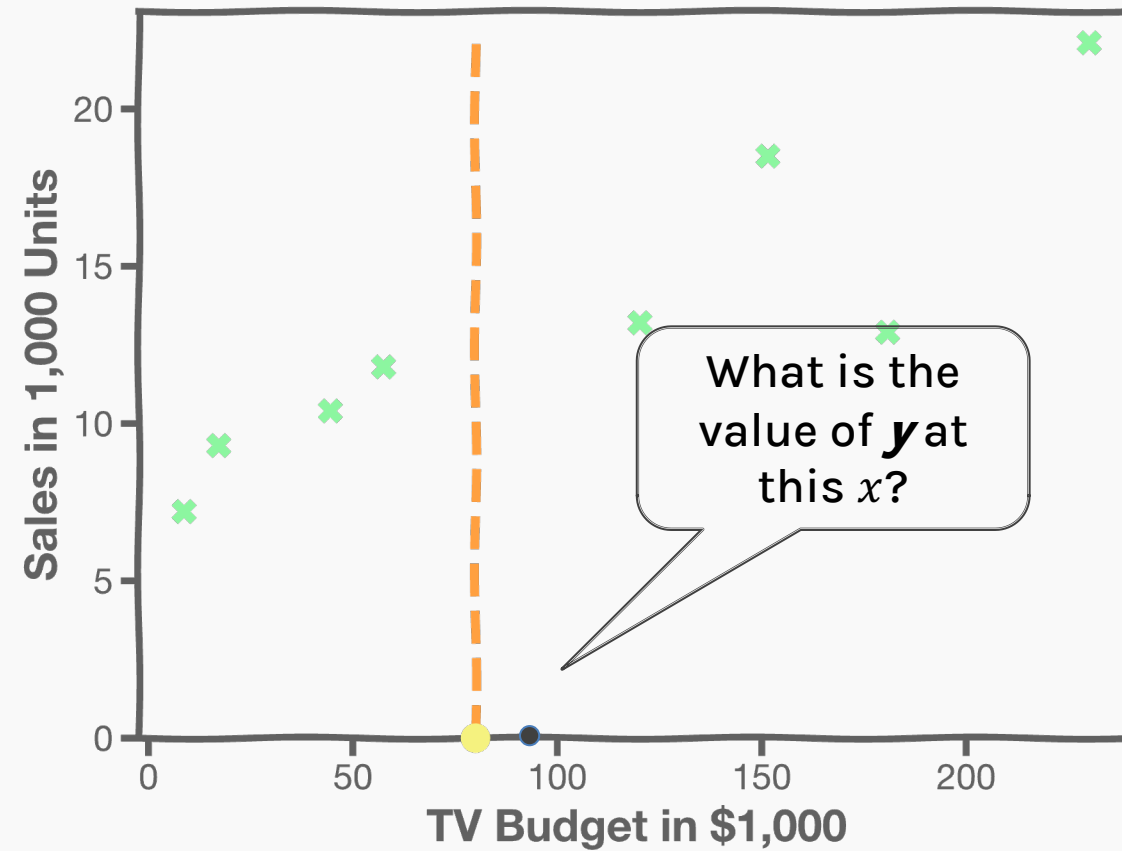
The predictor, **x**, is TV budget.

Statistical Model



Statistical Model

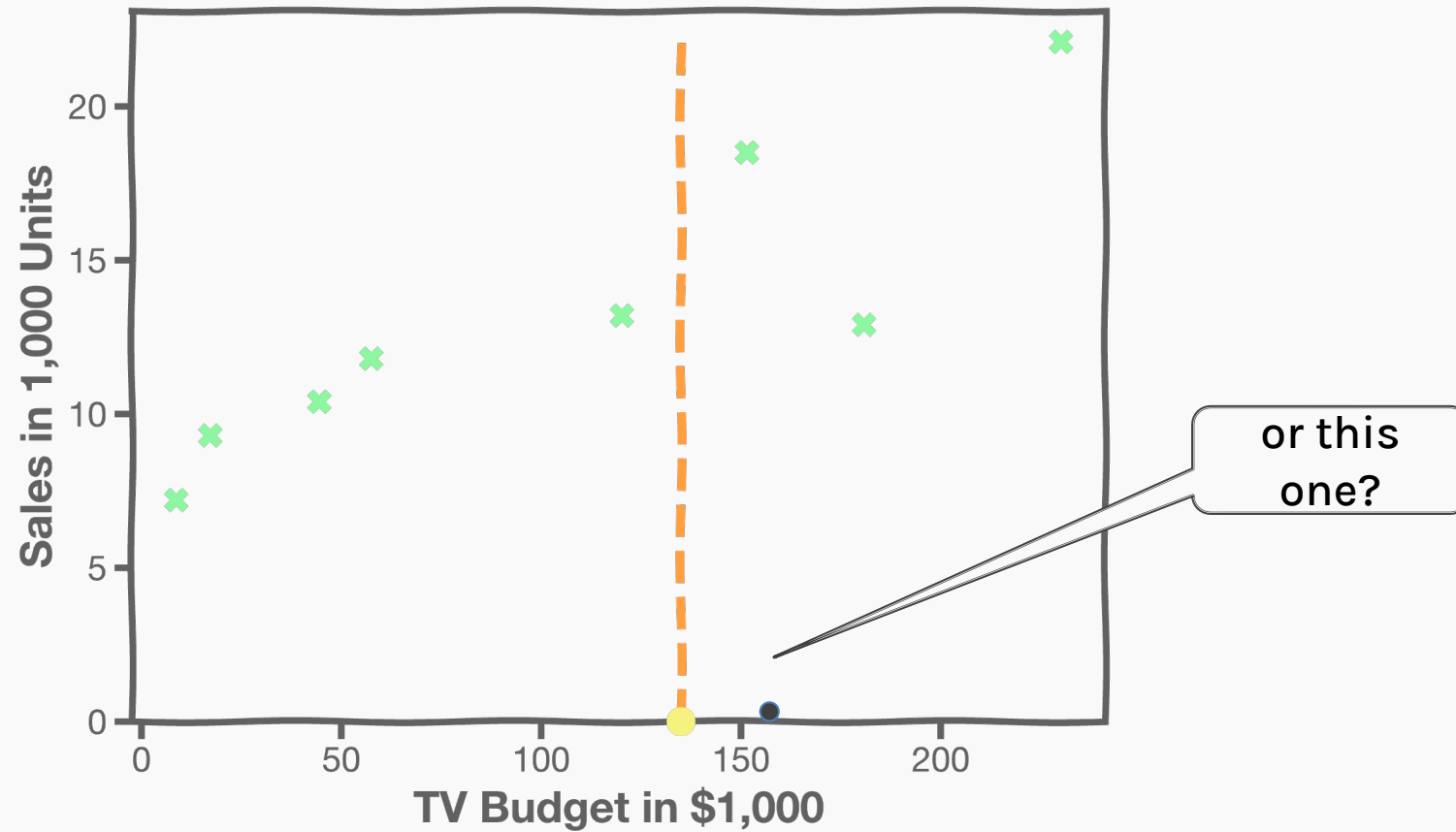
How do we predict y for some x ?



Statistical Model

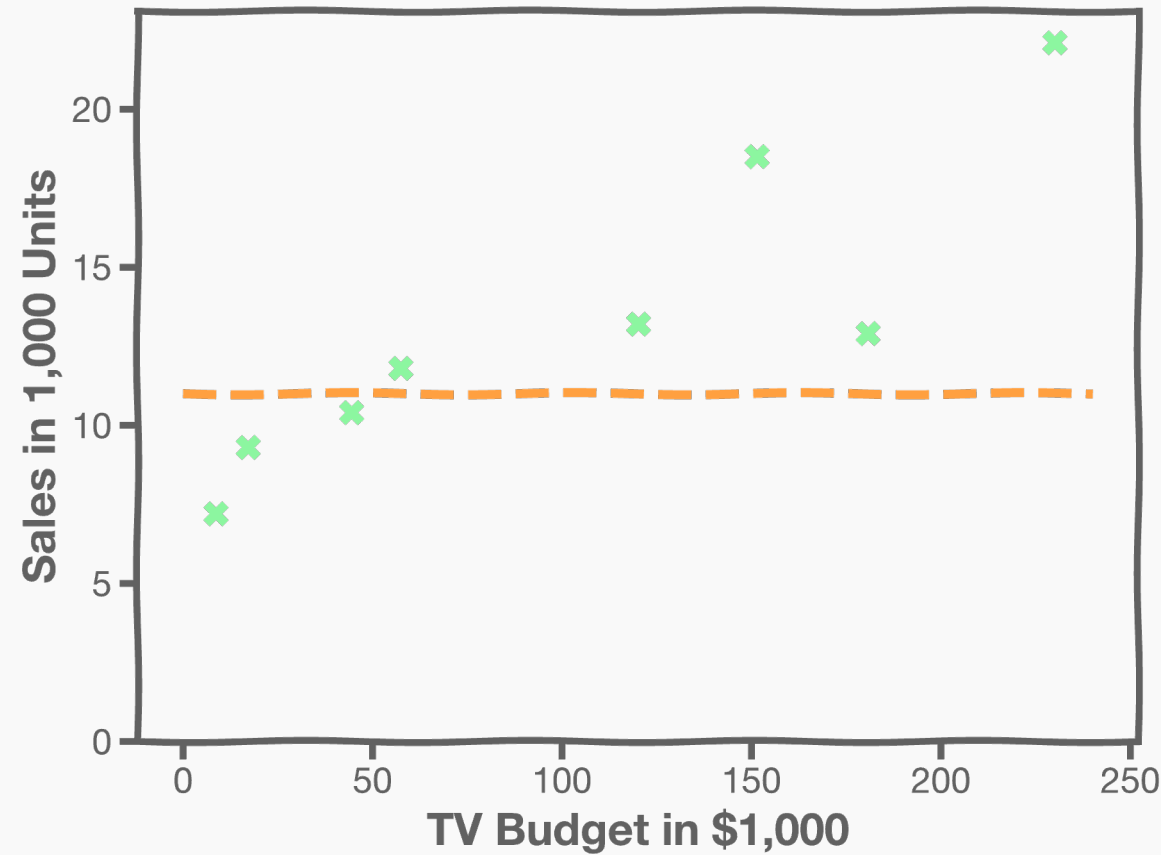


How do we predict y for some x ?



Statistical Model

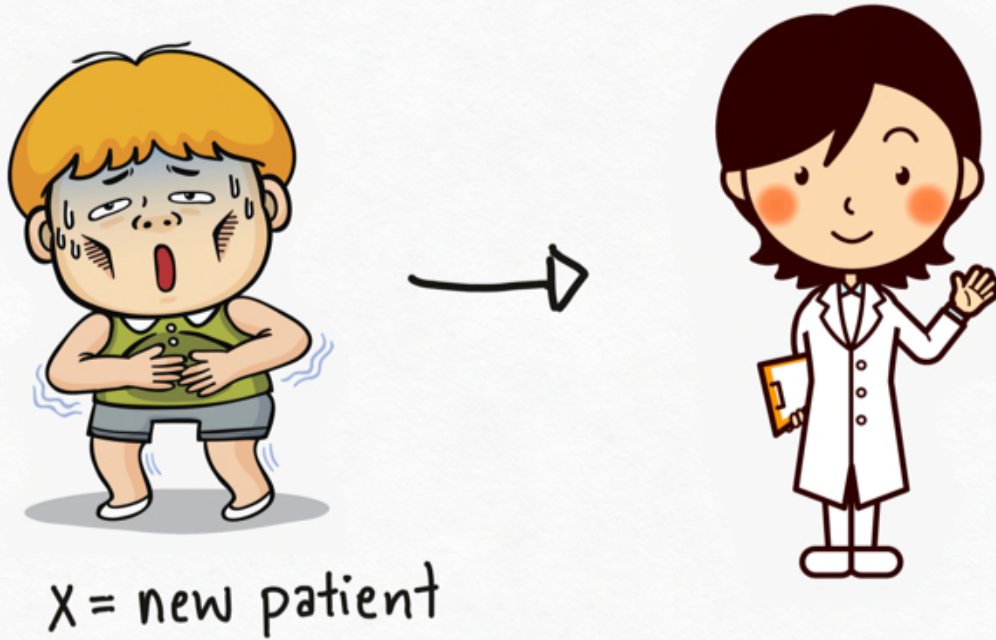
A simple idea is to take the mean of all y 's: $\frac{1}{n} \sum_{i=1}^n y_i$



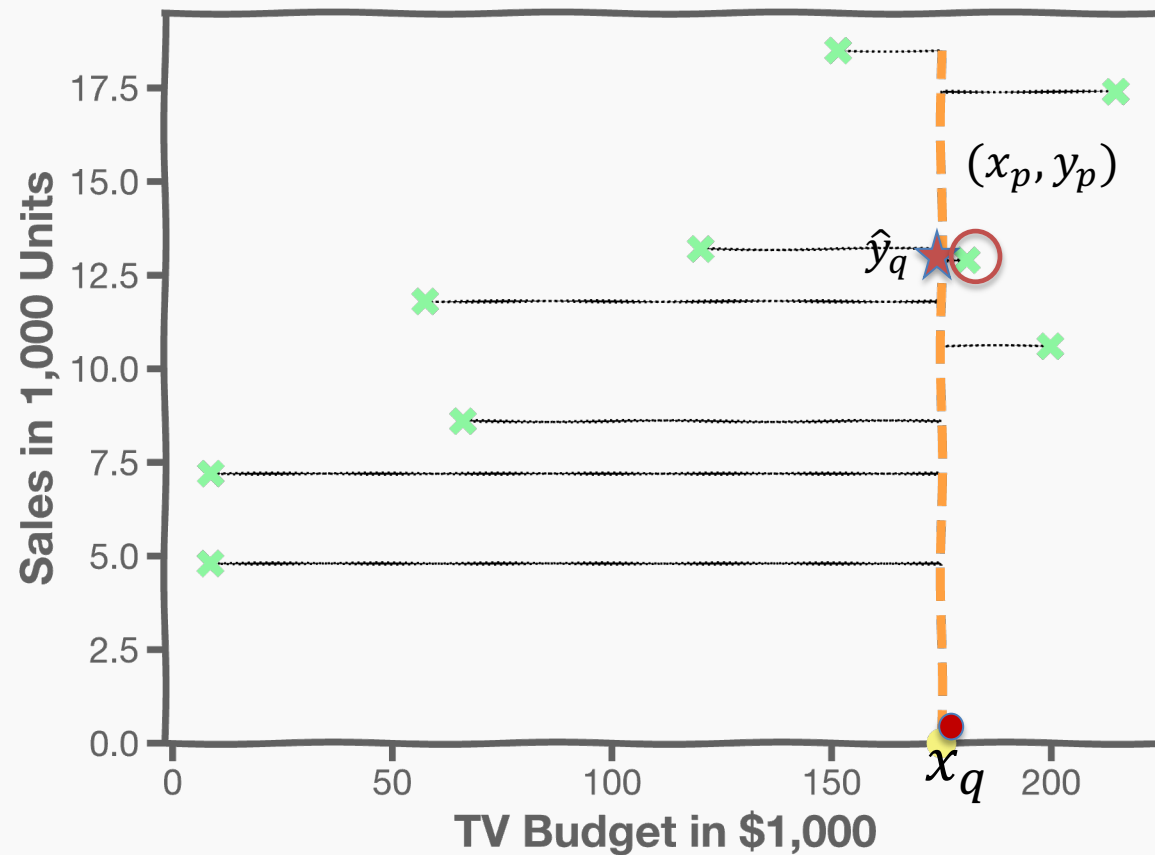
$$\frac{1}{n} \sum_{i=1}^n y_i$$

K-Nearest Neighbors

k-Nearest Neighbors – kNN



k-Nearest Neighbors - kNN



What is $\hat{y}_q$ at some x_q ?

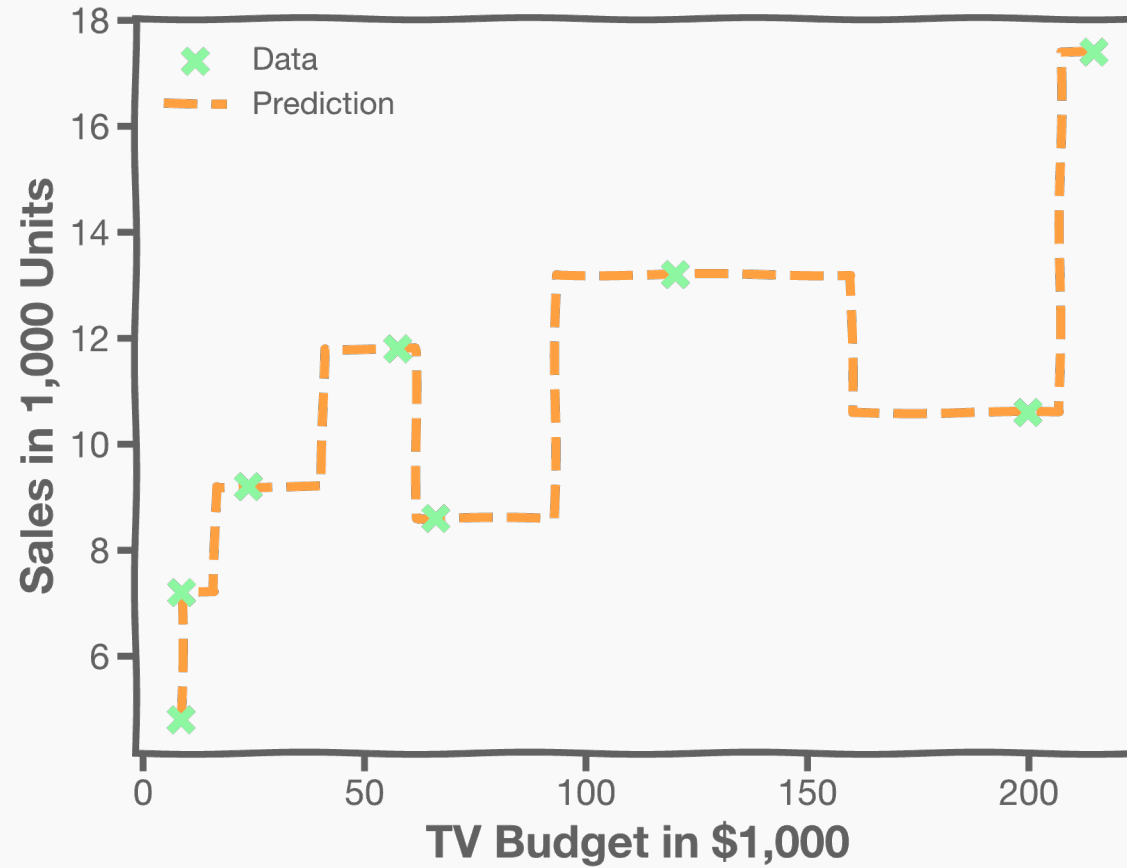
Find distances to
all other points
 $D(x_q, x_i)$

Find the nearest
neighbor, (x_p, y_p)

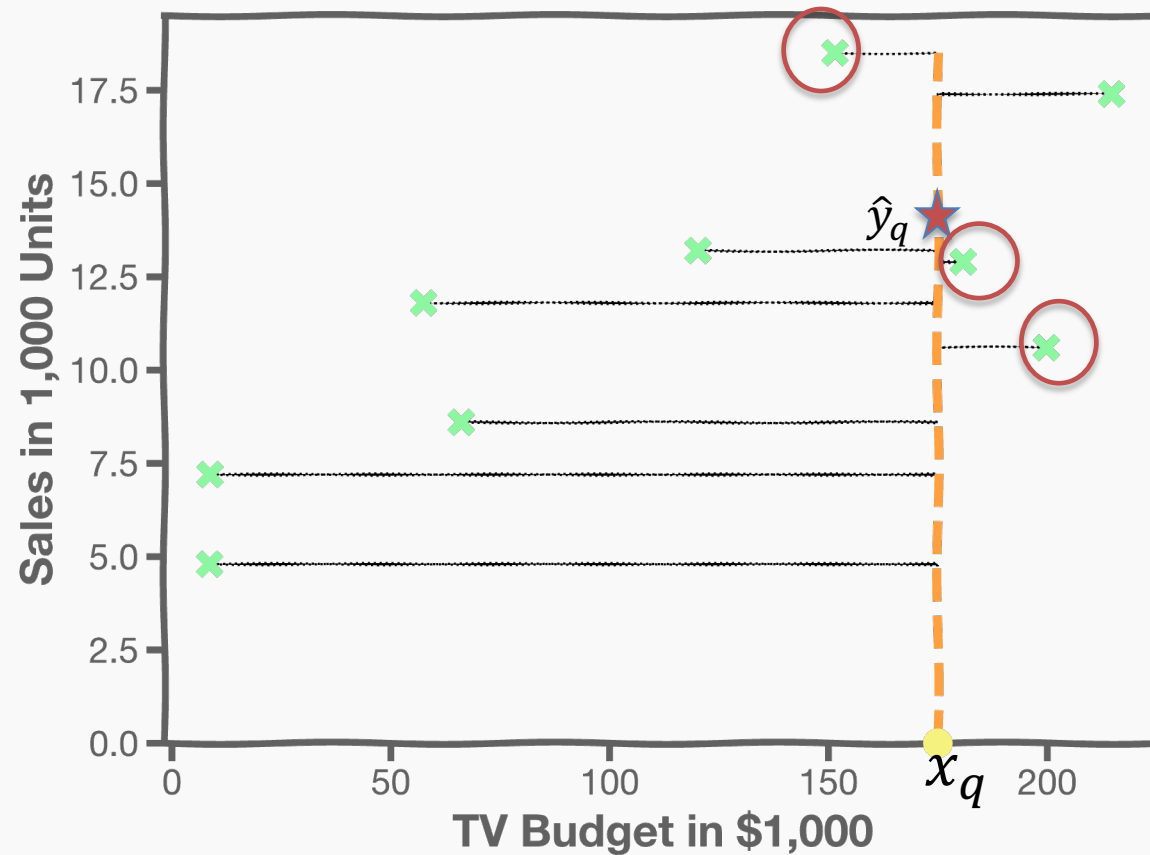
Predict $\hat{y}_q = y_p$

k-Nearest Neighbors - kNN

Do the same for “all” $x's$



k-Nearest Neighbors - kNN



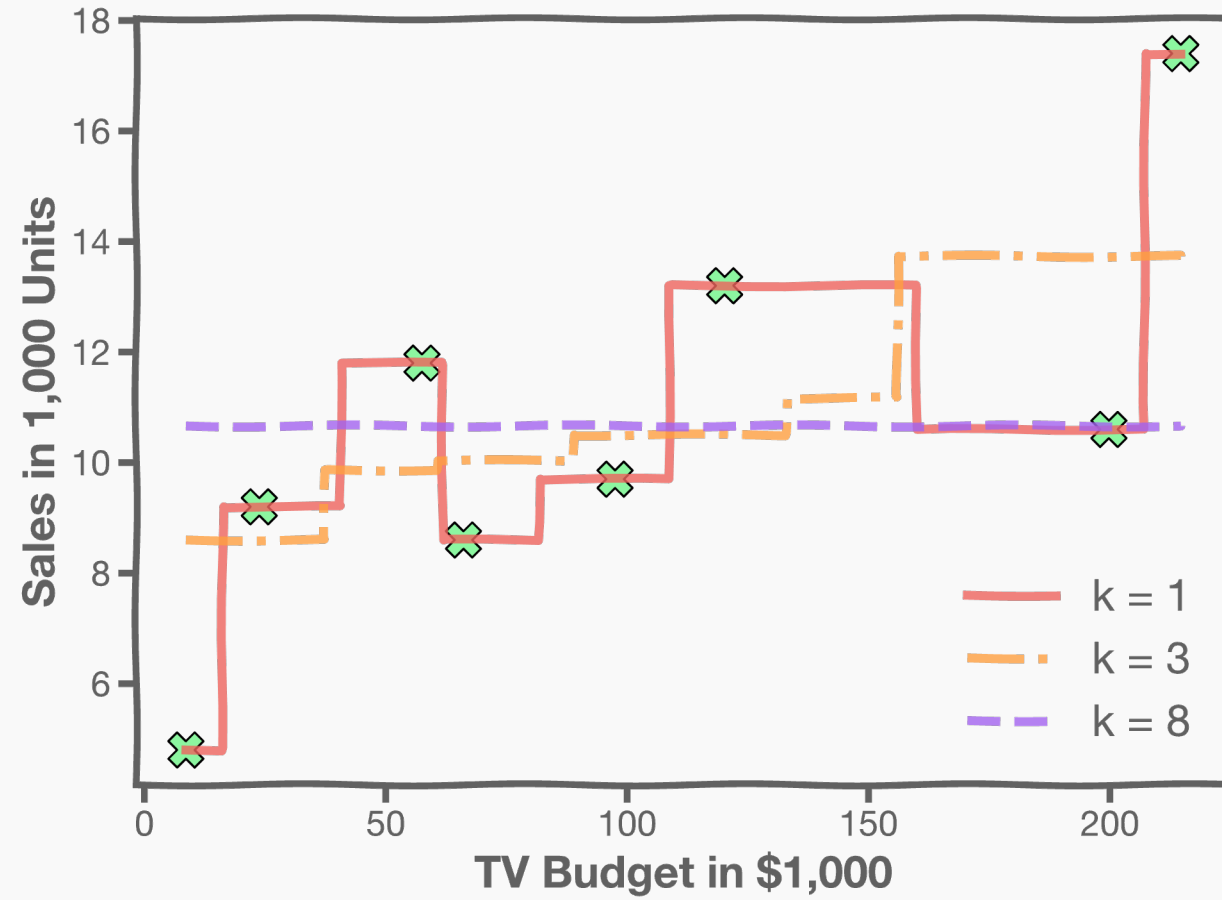
What is $\hat{y}_q$ at some x_q ?

Find distances to
all other points
 $D(x_q, x_i)$

Find the k-nearest
neighbors, $x_{q_1}, \dots, x_{q_k}$

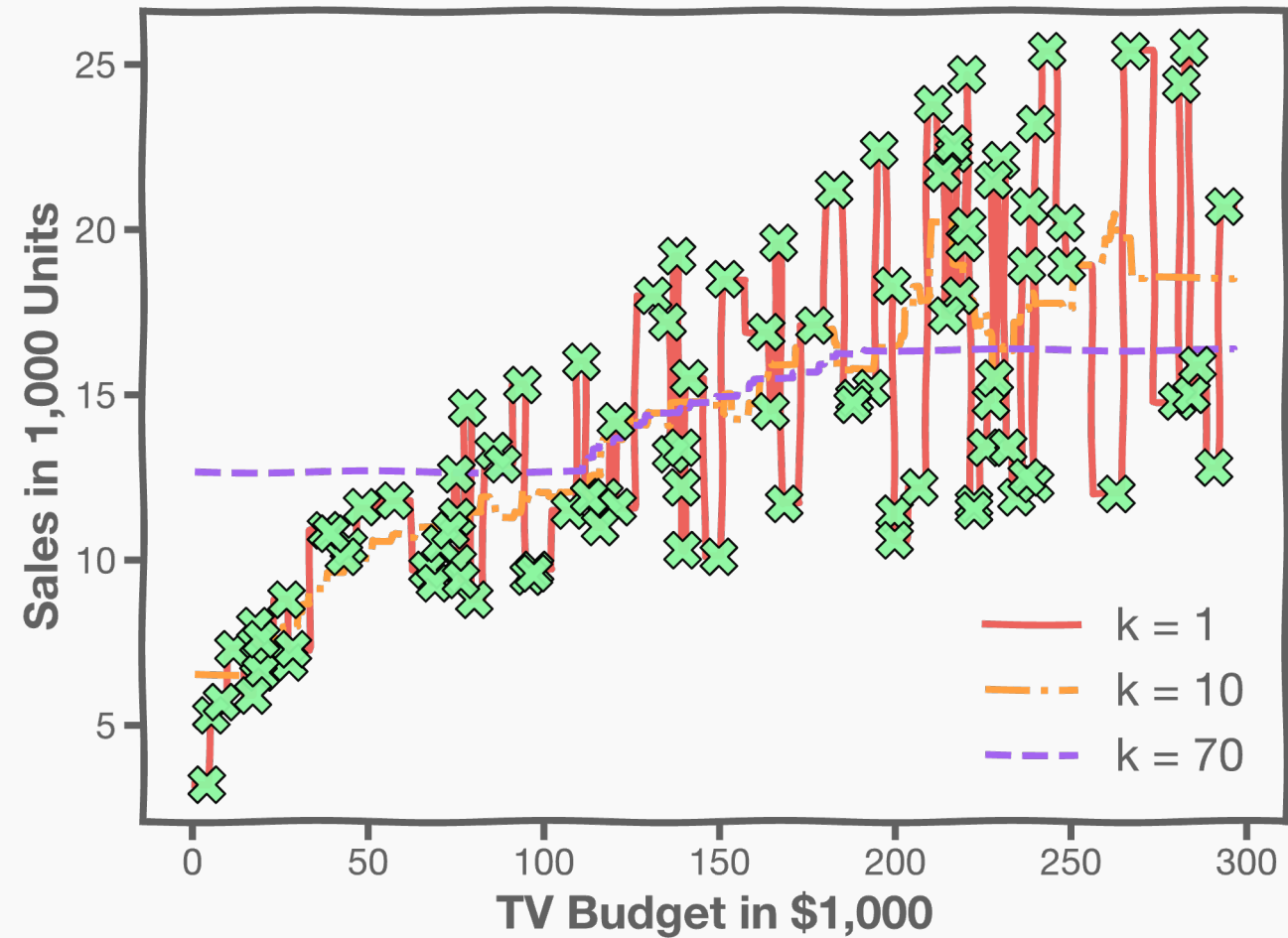
Predict $\hat{y}_q = \frac{1}{k} \sum_{i=1}^k y_{q_i}$

k-Nearest Neighbors - kNN



k-Nearest Neighbors - kNN

We can try different k-models on more data



k-Nearest Neighbors – kNN

The **very human way** of decision making by similar examples.

kNN is a **non-parametric** learning algorithm.

We call kNN non-parametric because it **doesn't assume** a **fixed form** for the **relationship** between **input** and **output**.

In simpler words, we **only rely on data** to get the prediction.

We will be looking at examples where we do assume a fixed form in the future lectures.



k-Nearest Neighbors – kNN

The **very human way** of decision making by similar examples.
kNN is a **non-parametric** learning algorithm.

The k-Nearest Neighbor Algorithm:

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})\}$, for every new X :

1. Find the k-number of observations in D most similar to X :

$$\{(x^{(n_1)}, y^{(n_1)}), \dots, (x^{(n_k)}, y^{(n_k)})\}$$

These are called the **k-nearest neighbors** of x

2. **Average** the output of the k-nearest neighbors of x

$$\hat{y} = \frac{1}{K} \sum_{k=1}^K y^{(n_k)}$$

k-Nearest Neighbors – kNN

$$\hat{y} = \frac{1}{K} \sum_{k=1}^K y^{(n_k)}$$

K here is what we call a **hyper-parameter**.

It is a parameter that **we choose** before the model training.

Remember:

- Non-parametric doesn't mean 'no hyperparameters'!
- It means the model **doesn't learn fixed parameters** from the data.

We will learn more about these hyper-parameters in the upcoming lectures!